Multimodal Insurance Creator

of a Multi-Agent Insurance System

Course Name: DevOps

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
01	Mustafa shaikh	EN22CS301624
02	Naman Mehta	EN22CS301629
03	Naman Raghuvanshi	EN22CS301632
04	Nandini Sharma	EN22CS301635
05	Nishkarsh Sharma	EN22CS301659

Group Name: DO-20

Project Number: 09D6

Industry Mentor Name: Rishab Sir

University Mentor Name: Prof. Ajaz Khan

Academic Year: 2025-2026

1. Introduction

The Multimodal GenAI Application for Insurance Concept Visualization is designed to generate both textual and visual representations of insurance-related concepts using Generative AI models. The system integrates Large Language Models (LLMs) for text generation and image generation models via APIs to produce descriptive narratives and corresponding visuals such as risk infographics.

The application focuses on transforming user input into structured outputs by orchestrating multiple AI components, enabling a unified and interactive representation of complex insurance concepts.

1.1 Scope of the Document

This document provides the Low-Level Design (LLD) of the system, detailing the internal components, module interactions, and processing logic involved in generating multimodal outputs.

The scope includes:
• Detailed description of system modules and components
• Interaction flow between LLM and image generation APIs
• Input processing and output generation logic
• Data handling and API communication
• Integration workflow for multimodal content generation

1.2 Intended Audience

Developers – To understand module-level implementation and integration logic
AI/ML Engineers – To analyze model interaction and API orchestration
System Designers – To understand component-level design and workflow
Testers – To identify functional flows for validation and testing
Project Evaluators – To review the technical implementation details

1.3 System Overview

The system operates by accepting a textual prompt related to an insurance concept from the user. The input is first processed by the backend service, which forwards the request to an LLM API for generating a detailed textual explanation.

Simultaneously, the same input is sent to an image generation API to produce a visual representation. The backend aggregates both outputs and formats them into a unified response.

The system consists of key modules including input handling, prompt processing, API integration layer, response aggregation, and output rendering. Each module performs a specific function to ensure seamless coordination between text and image generation, resulting in a complete multimodal output.

2. System Design

The system is designed as a modular and service-oriented architecture that integrates multiple AI components to generate multimodal outputs. It combines text generation using LLMs and image generation through external APIs to produce both descriptive and visual representations of insurance concepts.

The architecture ensures clear separation of concerns, where each module handles a specific responsibility such as input processing, API communication, and response aggregation. The system supports parallel processing of requests to optimize performance and reduce latency.

The design includes a frontend interface for user interaction and a backend service that orchestrates communication between different components. External AI services are integrated through APIs, allowing flexibility and scalability in model selection.

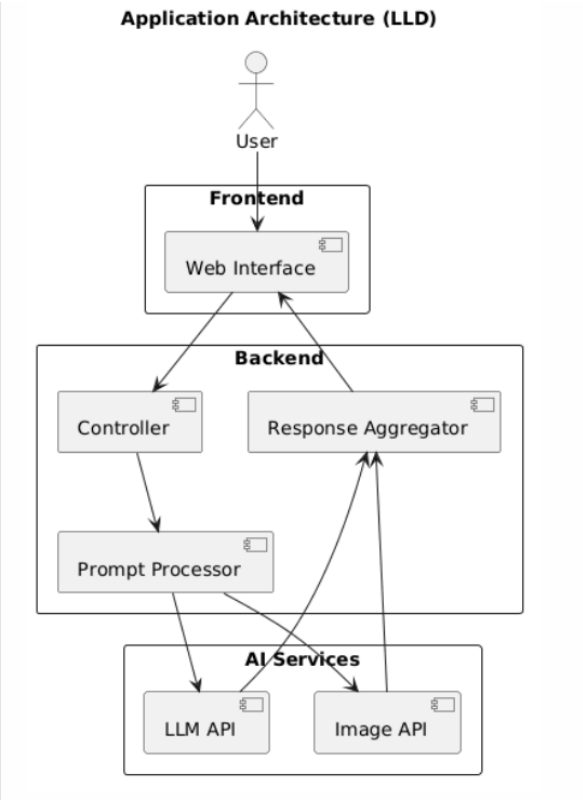
The overall system workflow involves capturing user input, processing it through AI models, aggregating the outputs, and presenting the final result in a unified format. The modular design enables easy enhancement, maintenance, and integration of additional features in the future.

2.1 Application Design

The application is divided into frontend, backend, and AI service layers. The frontend handles user interaction, while the backend processes requests and communicates with AI APIs. The system ensures separation of concerns for maintainability.

Components:

• User Interface (Input/Output)
• Backend Controller
• Prompt Processing Module
• LLM Integration
• Image Generation Integration
• Response Aggregator

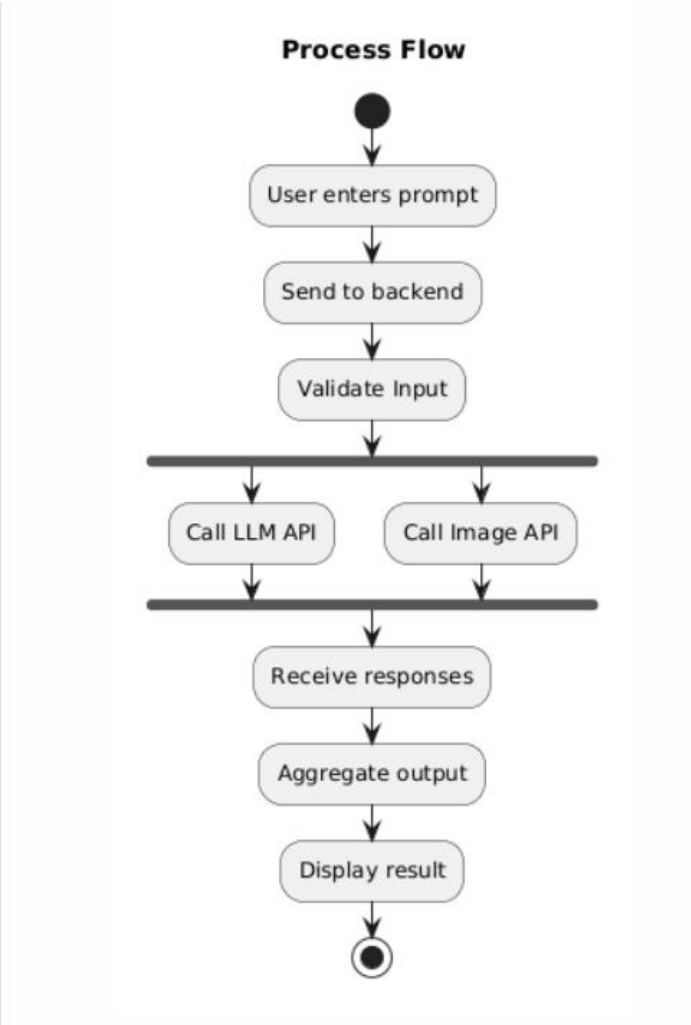


1.2. Process Flow

The process starts with user input, followed by backend processing and parallel API calls. The outputs are combined and displayed.

Steps:

• User submits prompt
• Input is validated and processed
• Parallel calls to LLM and Image API
• Responses are collected
• Output is aggregated and displayed



1.3 Information Flow

Information flows from user input to backend processing and then to external AI services. The responses are returned and combined before presentation.

Flow:

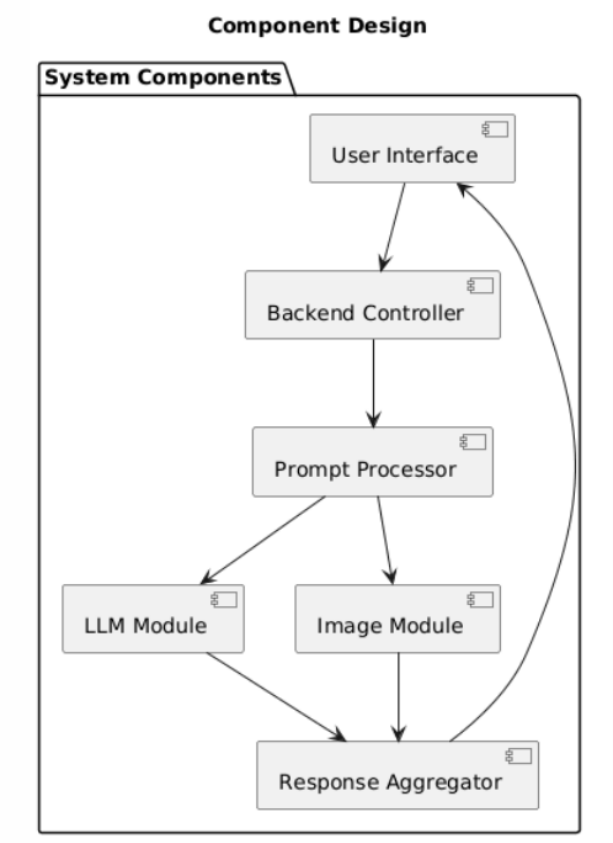
• User → Frontend
• Frontend → Backend
• Backend → AI APIs
• APIs → Backend
• Backend → Frontend

1.4 Components Design

The system is composed of multiple modules, each responsible for a specific function.

Components:

• **User Interface** – Captures input and displays results
• **Backend Controller** – Handles requests and routing
• **Prompt Processor** – Prepares input for APIs
• **LLM Module** – Generates textual output
• **Image Module** – Generates visual output
• **Response Aggregator** – Combines outputs
• **Renderer** – Displays final result



1.5 Key Design Considerations

The system is designed with focus on performance, scalability, and flexibility.

Considerations:

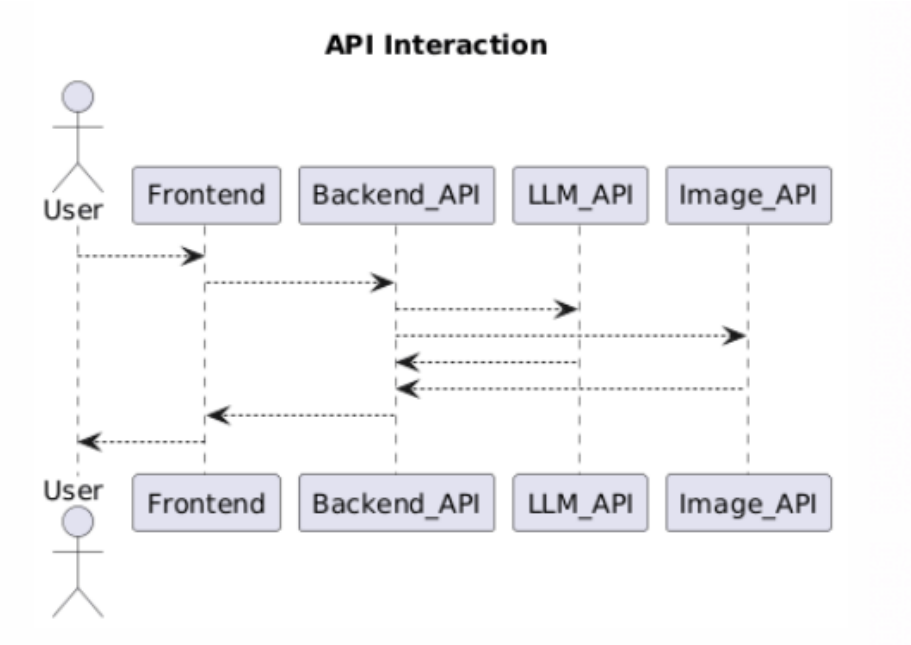
• **Modularity** – Independent components for easy updates
• **Scalability** – Supports adding new AI models
• **Performance** – Parallel API execution
• **Reliability** – Error handling for API failures
• **Extensibility** – Future support for audio/video models
• **Security** – Safe API communication

1.6 API Catalogue

The system uses APIs for communication between components and external services.

APIs:

• **LLM API** – Generates textual descriptions
• **Image Generation API** – Creates visual outputs
• **Backend API** – Handles frontend-backend communication



8. References

• OpenAI API – https://platform.openai.com/docs
• Flask – https://flask.palletsprojects.com/
• Stable Diffusion – https://stability.ai/
• JSON – https://www.json.org/

- Problem Statement: **Multimodal Insurance Creator**
- Project Group: **D6 - Group 09**.